MAGIVOX AI

Complete Code, Architecture & Interview Guide

A beginner-friendly technical guide based on the actual current project files

HTML, CSS, JavaScript, Firebase, Firestore, Gemini AI, courses, assessments, voice interview, resume, jobs, notes, portfolio and admin

Document basis	Verified current project
Source inspected	Root HTML/CSS/JavaScript modules, Firebase rules, Cloud Functions, tests and build scripts
Application source changes	None - this document is generated separately
Audience	Project owner preparing to understand, explain and interview on the code
Generated	14 September 2026

How to use this guide

Read Parts 1-7 first for the big picture. Then study the feature chapter you need. Use Part 23 as a learning schedule and Part 24 for interview practice. The code excerpts are intentionally short; each one names its real source file so you can find the complete implementation.

Table of Contents

Table of Contents 2

Part 1: Project Overview and Architecture 5

- Architecture at a glance . . . 5
- Why it was built and who it serves . . . 5
- Full learner journey . . . 5

Part 2: Folder and File Structure 6

- Important file connection catalog . . . 6
- Dependency direction . . . 7

Part 3: HTML Foundations in This Project 8

- Static shell and dynamic tool content . . . 8
- Basic HTML document anatomy . . . 8
- Accessible form structure . . . 8
- Common attributes . . . 9

Part 4: CSS and Responsive Design 10

- Actual sidebar selector, line by line . . . 10
- Actual responsive rule . . . 10
- Grid example . . . 10
- Layout model . . . 10
- How to diagnose spacing . . . 11

Part 5: JavaScript Basics Used by MAGIVOX 12

- Core language concepts in this project . . . 12
- ES modules . . . 12
- DOM event pattern . . . 12
- Async pattern . . . 12
- A function from input to UI . . . 13

Part 6: Important Functions and Responsibilities 14

- Function contracts: input, output and dependencies . . . 14

Part 7: Event Flow: From Button to Visible Result 16

- General event sequence . . . 16
- Worked example: Save resume version . . . 16
- Major event flows . . . 16

Part 8: Firebase Authentication 18

- Email and Google login . . . 18
- Profile-aware redirect . . . 18

Password security . . . 18

Part 9: Firestore Data Model and Persistence **19**

Primary document model . . . 19
Public and operational collections . . . 19
Firestore verbs in plain English . . . 19

Part 10: Firestore Security Rules **20**

Owner and admin checks . . . 20
Security decisions . . . 20

Part 11: Shared MAGIVOX AI Core and Gemini **21**

One operation registry . . . 21
Firebase AI Logic client . . . 21
Quota and failure handling . . . 21

Part 12: Course and Lesson Flow **22**

Required sequence . . . 22
What one lesson contains . . . 22

Part 13: Exams, Aptitude and Reasoning **23**

Course assessments . . . 23
Aptitude engine . . . 23
Integrity of scores . . . 23

Part 14: ATS Resume Studio **24**

Two workflows . . . 24
Readiness gate . . . 24
AI safety . . . 24
When Gemini is unavailable . . . 24

Part 15: Voice-First Interview and English Coach **25**

Modes and session model . . . 25
Natural voice cycle . . . 25
Speech accuracy pipeline . . . 25
Multi-turn report . . . 25

Part 16: Progress, Portfolio and Readiness **26**

Overall readiness weights . . . 26
Public portfolio . . . 26

Part 17: Career Roadmaps and Mentor Plans **27**

Roadmap evidence . . . 27
Milestones . . . 27

Part 18: Jobs and Application Tracking 28

Evidence match formula . . . 28

Safety and tracking . . . 28

Part 19: Search, Saved Items, Notes, Flashcards and Notifications 29

Unified local search . . . 29

Notes and AI . . . 29

Notifications . . . 29

Part 20: Admin System: Current Capability and Limits 30

Implemented . . . 30

Honest limitation . . . 30

Part 21: Error Handling and Recovery 31

Retry principle . . . 31

Part 22: Dependency and Data-Flow Diagrams 32

Authenticated workspace . . . 32

AI request . . . 32

Course certificate . . . 32

Interview answer . . . 32

Job preparation . . . 32

Module dependency table . . . 32

Part 23: Beginner Reading and Learning Plan 33

How to explain any feature . . . 33

Part 24: Project-Specific Interview Questions and Answers 34

Part 25: How to Describe Your Role Honestly 39

Recommended answer . . . 39

If asked what you coded . . . 39

Evidence you can show . . . 39

Part 26: Complete Code Summary, Major Buttons and Top Files 40

Every major button: code-to-result map . . . 40

Complete important-file summary . . . 42

Verified quality state while preparing this guide . . . 43

Top 20 files to understand first . . . 43

Final architecture summary . . . 43

Part 1: Project Overview and Architecture

MAGIVOX AI is a browser-based learning and career platform. Static HTML pages load ES modules. Firebase Authentication identifies the user. Firestore stores profile and learning state. Deterministic JavaScript engines calculate progress, scores and readiness. Gemini, reached through Firebase AI Logic, handles genuine generative conversations and coaching.

Architecture at a glance

Browser pages -> **Feature mount function** -> **Deterministic engine** -> **Firestore or AI Core** -> **Rendered result**

Layer	Real files	Responsibility
Pages	index.html, login.html, signup.html, dashboard.html, workspace.html, admin.html	Semantic shells, forms, navigation and containers
Presentation	style.css, workspace.css, studio.css	Responsive layout, cards, forms, states and print presentation
Orchestration	workspace.js, studio.js, dashboard-data.js	Auth guard, state loading, tool routing and feature mounting
Feature UI	teacher.js, interview-studio.js, resume-builder.js, jobs.js, platform-hub.js	Build tool interfaces and bind events
Domain engines	course-learning-engine.js, interview-engine.js, resume-engine.js, job-engine.js, portfolio-engine.js	Pure normalization, rules, scoring and recommendations
Cloud services	firebase-auth.js, learning-store.js, resume-store.js, portfolio-store.js, ai-client.js	Authentication, persistence, AI transport and public records
Security/backend	firestore.rules, functions/*.js, firebase.json	Authorization, validation, routes, quotas and hosting rewrites

Key boundary: calculations such as ATS scoring, eligibility and progress are local and deterministic. Text described as AI-generated goes to the hosted Gemini provider. If quota fails, the user's input and non-AI tools remain available.

Why it was built and who it serves

MAGIVOX was built to give beginners one guided path from learning a skill to showing job evidence. The primary users are students, fresh graduates, career switchers and early-career learners in India who need structured theory, practice, interview communication and application preparation in one account.

Full learner journey

Create account -> **Complete profile** -> **Choose career role** -> **Learn course** -> **Practice and test**

Earn certificate -> **Build resume/project** -> **Practise interview** -> **Prepare job** -> **Track progress**

- The profile supplies display name, preferred language, target role and experience level.
- The roadmap recommends role-relevant courses but never awards evidence from a recommendation alone.
- Course work creates assessed evidence through lessons, practice and tests.
- Resume, projects, interview and aptitude add separate evidence to readiness.
- Jobs compare supplied requirements with evidence; the learner controls every application.

Part 2: Folder and File Structure

The project intentionally keeps production modules at the repository root. `dist-pages/` is generated by the build and mirrors deployable public files. `functions/` contains an optional server API path. `resources/` contains course PDFs and the Excel practice workbook.

Area	Important files	What to learn
Entry pages	index.html, login.html, signup.html, profile-setup.html	Public landing, account creation and onboarding
Authenticated shells	dashboard.html, workspace.html, workspace.js	Dashboard and query-string tool router
Learning	course-catalog.js, lesson-curriculum.js, course-learning-engine.js, assessment-engine.js	Catalog data, lesson theory, gates and exams
AI	ai-core.js, ai-client.js, teacher.js, teacher-conversation.js	One operation registry, Gemini client, context and teacher UI
Career	career-roadmap-engine.js, job-engine.js, resume-engine.js, portfolio-engine.js	Evidence-driven roadmap, job, resume and readiness logic
Persistence	learning-store.js, resume-store.js, portfolio-store.js	Narrow Firestore functions
Quality	test/*.test.js, functions/test/*.test.js, scripts/check.mjs, scripts/build.mjs	Unit checks, syntax checks and deploy copy
Generated	dist-pages/	Build output; edit source files instead

Important file connection catalog

File	Why it exists	Feature	Key functions	Connects to
workspace.js	Authenticated application coordinator	Courses and every workspace tool	renderActiveTool, showCourse, changeWorkspaceState	workspace.html, engines, stores, Firebase
workspace.html	Stable workspace document	Sidebar, search, notifications, tool container	Static HTML IDs/data attributes	workspace.js, workspace.css, firebase-auth.js
workspace.css	Feature layout rules	Course, aptitude, jobs, portfolio, responsive UI	Selectors and media queries	workspace.html and dynamic feature markup
studio.js	Studio export facade	Teacher, resume review, interview	Re-exports mount functions	teacher.js, resume reader UI, interview-studio.js
ai-core.js	Shared generative-AI boundary	All hosted AI features	conversationHistory, requestMAGIVOXAI	ai-client.js and feature UIs
ai-client.js	Gemini transport and schemas	Teacher, interview, resume, career, exam, transcription	requestAI, friendlyAIError, modelFor	firebase-auth.js, Firebase AI Logic, engines
firebase-auth.js	One Firebase app/auth instance	Login, signup, logout, reset, auth guards	redirectAfterLogin, auth event handlers	Firebase SDK and all authenticated pages
learning-store.js	Narrow Firestore learning API	Teacher library, support, certificates	saveLearningItem, submitSupport, publishCertificate	firebase-auth.js, firestore.rules
course-catalog.js	Static course/topic metadata	Skills, search, exams, roadmaps	getCourse, courseSearchText	workspace, teacher, job and roadmap engines
lesson-curriculum.js	Detailed learning matter	Beginner-through-project lesson sections	getTopicCurriculum	course catalog and workspace lesson renderer
course-learning-engine.js	Course state machine	Progress, unlocks, final and certificate	recordPractice, completeLesson, courseEligibility	workspace, portfolio, Firestore state
assessment-engine.js	Assessment runtime	Topic and 50-question final exams	mountAssessmentCenter	course engine, AI Core, workspace
interview-studio.js	Voice conversation coordinator	Introduction, mock interview, English coach	mountInterviewStudio, submit, speakThenListen	voice-recorder, interview engine, AI Core
speech-normalizer.js	Safe STT validation	Interview microphone transcript	normalizeSpeechTranscript, assessSpeechReliability	interview studio and tests
resume-builder.js	Structured resume UI	Editor, versioning, export, AI assist	mountResumeBuilder, draw, printResume	resume engine/store, AI Core
resume-engine.js	Resume rules	ATS, readiness, JD match, normalization	analyzeResume, analyzeJobMatch	resume builder, portfolio/job engines

File	Why it exists	Feature	Key functions	Connects to
portfolio-engine.js	Cross-feature evidence aggregator	Readiness, skills, badges, public payload	buildPortfolioEvidence, publicPortfolioPayload	course/resume/aptitude engines
career-roadmap-engine.js	Role planning engine	Gaps, milestones, daily/weekly plans	careerEvidence, smartRecommendation	catalog, portfolio engine, roadmap UI
job-engine.js	Job rules	Search, match, safety, preparation, tracking	calculateJobMatch, preparationPack	catalog, portfolio, jobs UI
platform-hub.js	Library UI	Search, saved, notes, flashcards, notifications	mountPlatformHub, noteAI	platform engine, AI Core, workspace state
admin.js	Admin-only browser console	Metrics, content registry, request status	load, drawItems, edit	Firestore and Firebase custom claims
firestore.rules	Server-side data authorization	All Firestore access	owner, admin, validation helpers	Every Firestore read/write

Dependency direction

UI module -> **Domain engine** -> **Shared service** -> **Firebase SDK**

A good debugging habit is to start at the visible button, locate its handler in the feature UI module, follow imported engine calls, and finish at either a Firestore store or `requestMAGIVOXAI`.

Part 3: HTML Foundations in This Project

Static shell and dynamic tool content

`workspace.html` owns the fixed sidebar, top bar and the empty tool container. Most feature screens are inserted later with `container.innerHTML` inside mount functions. This keeps navigation stable while each tool controls its own form and buttons.

Basic HTML document anatomy

Verified source: **login.html (shortened to the real structural pattern)**

```
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
  <link rel="stylesheet" href="style.css">
</head>
<body>
  <main>...</main>
  <script type="module" src="firebase-auth.js"></script>
</body>
</html>
```

Line/element	Simple meaning	Why MAGIVOX needs it
DOCTYPE	Tells the browser to use modern HTML	Prevents old compatibility rendering
html lang=English	Root element and document language	Helps browsers and assistive tools
head	Page settings, title, metadata and style links	Loads responsive settings and CSS before content
body	Everything visible or interactive	Contains login/workspace interface
main/section/div	Semantic main area, grouped section, generic layout box	Builds clear page and card hierarchy
script type=module	Loads a JavaScript ES module	Allows import/export in Firebase and feature code

Verified source: **workspace.html**

```
<a class="nav-link" data-tool-link="interview"
   href="workspace.html?tool=interview">Interview Practice</a>
<div id="workspaceTool"></div>
```

Line 1: `a` creates a link. The `nav-link` class gives shared navigation styling. `data-tool-link` lets JavaScript find and highlight the active tool.

Line 2: `href` reloads the workspace with `tool=interview`; the visible words tell the user where it goes.

Line 3: the unique `workspaceTool` ID gives `workspace.js` one target into which it inserts the selected feature.

Accessible form structure

The project usually pairs labels with controls, gives status areas `role="status"`, gives conversations `role="log"`, and uses actual buttons for actions. `type="button"` prevents accidental form submission for secondary controls.

Verified source: **help-support.js (dynamic HTML)**

```
<form id="helpForm">
  <label for="helpInput">What do you need help with?</label>
  <textarea id="helpInput" required></textarea>
  <button id="helpSend">Get help</button>
  <p id="helpStatus" role="status"></p>
</form>
```

Line 1: the form groups one submission and its unique ID is used for the submit handler. **Line 2:** the label names the textarea and its `for` value matches the textarea ID. **Line 3:** `required` blocks an empty submission. **Line 4:** the button submits the form. **Line 5:** JavaScript writes progress or errors into a screen-reader-friendly status region.

Common attributes

Attribute	Purpose	Example
id	Unique lookup for a specific control	#endInterview
data-*	Repeatable action metadata	data-help, data-course, data-job-view
required / maxlength	Browser-side input boundary	support description max 4000
aria-live / role	Announce changing state	interview transcript and status
hidden / disabled	Control availability and staged flows	locked topics and busy AI requests

Part 4: CSS and Responsive Design

Three main style sheets divide responsibility: `style.css` covers public/auth pages and shared visual tokens, `workspace.css` covers the authenticated shell and feature layouts, and `studio.css` covers AI teacher, resume and interview workspaces.

Actual sidebar selector, line by line

Verified source: **style.css**

```
.sidebar {
  position: fixed;
  z-index: 20;
  inset: 0 auto 0 0;
  display: flex;
  width: 270px;
  padding: 28px 20px 22px;
  flex-direction: column;
  color: #d5e2f2;
  background: linear-gradient(180deg, var(--navy-950), #071c35 58%, #0b2749);
}
```

Property	Effect on screen
.sidebar	Selects every element whose class contains sidebar
position: fixed	Keeps desktop navigation attached to the viewport
z-index: 20	Places it above normal page content
inset	Pins top, bottom and left; right remains automatic
display: flex / direction column	Stacks brand, navigation and account vertically
width/padding	Sets the sidebar width and inner spacing
color/background	Sets readable light text on the navy gradient

Actual responsive rule

Verified source: **style.css**

```
@media (max-width: 980px) {
  .sidebar {
    position: static;
    width: 100%;
    padding: 16px 18px;
  }
  .sidebar-nav {
    display: flex;
    overflow-x: auto;
  }
}
```

When the screen is 980 px or narrower, the sidebar returns to normal document flow, spans the page and changes navigation into a horizontally scrollable row. This prevents the fixed 270 px desktop rail from crushing mobile content.

Grid example

Verified source: **workspace.css**

```
.resume-builder .studio-two-columns {
  column-gap: 16px;
  row-gap: 14px;
}
```

The combined selector targets `.studio-two-columns` only inside the resume builder. Separate column and row gaps keep paired fields aligned without placing margin on every input.

Layout model

Pattern	Where used	Reason
CSS Grid	dashboard cards, studio panels, jobs and resume workspace	Aligns related panels and collapses for narrow screens
Flexbox	toolbars, action rows, navigation	Handles variable button and label widths
Cards/panels	most tools	Clear visual grouping with border and spacing
Media queries	workspace and studio	Stack columns and preserve readable controls on mobile
Print rules	resume/certificate/portfolio print helpers	Create focused printable output

How to diagnose spacing

- Inspect the parent grid first: column width, gap and alignment.
- Check fieldset/legend padding because legends can overlap rounded borders.
- Check fixed or sticky elements against viewport height.
- At 320 px width, verify document width, horizontal overflow and button wrapping.

Interview explanation: CSS changes presentation only. The click behavior and data rules remain in JavaScript modules.

Part 5: JavaScript Basics Used by MAGIVOX

Core language concepts in this project

Concept	MAGIVOX example	Meaning
const	`const db = getFirestore(app)`	A binding that is not reassigned
let	`let messages = [], busy = false`	State that changes while the feature runs
string	`'English'` or a template literal	Text shown, saved or sent
number	Score 70 or question index 0	Numeric calculation and indexing
array	COURSES, messages, attempts	Ordered list; indexes begin at zero
object	`{ role:'user', content }`	Named fields grouped into one value
function	`courseEligibility(course, record)`	Reusable behavior with inputs and output
arrow function	`item => item.complete`	Short function passed to map/filter
if/else	Check user, consent, busy or phase	Runs different paths from a condition
loop	`for (const field of config.fields)`	Processes each allowed AI field
return	Returns normalized data or exits early	Gives a value back or stops a function
async/await	Await Firebase, AI and save promises	Writes asynchronous code in readable order
try/catch/finally	AI and Firestore handlers	Handles failure and always releases busy state
import/export	Shared engine functions	Connects modules without global variables
DOM	`document.getElementById`	JavaScript representation of page elements
event	click, submit, input, change	A user/browser action that triggers code
localStorage	Guest identity and small preferences	Browser-only key/value persistence
JSON	Structured payloads and app data	Text representation of arrays/objects
Promise	Future result of Firebase/AI work	Can resolve successfully or reject with an error

ES modules

Verified source: **teacher.js**

```
import { requestMAGIVOXAI } from './ai-core.js';
export function mountTeacher(context) { /* build and bind UI */ }
```

Imports make dependencies explicit. Exported engine functions can be tested without opening the browser. Feature mount functions receive a context object instead of importing every part of workspace state.

DOM event pattern

Verified source: **jobs.js**

```
el('#jobsFilterForm').onsubmit = event => {
  event.preventDefault();
  drawResults();
};
```

Async pattern

Verified source: **Repeated feature pattern**

```
try {
  const result = await requestMAGIVOXAI(operation, payload, user);
  target.textContent = result.answer;
} catch (error) {
  target.textContent = error.message;
} finally {
  busy = false;
}
```

The `busy` flag blocks duplicate clicks. `try/catch/finally` guarantees that controls are restored after success or failure.

A function from input to UI

Verified source: **speech-normalizer.js (body shortened)**

```
export function normalizeSpeechTranscript(rawTranscript, { profileName = '' } = {}) {
  const raw = String(rawTranscript || '').trim();
  let corrected = raw;
  // guarded name and technical-term corrections run here
  return { raw, corrected, changes };
}
```

- Input: browser transcript plus an optional saved profile name.
- Inside: convert input to safe text, preserve raw, apply only guarded corrections and record changes.
- Return: an object containing raw, corrected and changes.
- Called from: the interview recording/transcription flow.
- UI afterward: raw, final and validated text can be shown; only a reliable corrected final transcript is submitted.

Part 6: Important Functions and Responsibilities

File	Function	Responsibility
workspace.js	renderActiveTool	Reads `tool` from the URL and mounts the selected feature.
ai-core.js	requestMAGIVOXAI	Scopes payload fields, limits history, prevents duplicate in-flight requests and normalizes quota errors.
ai-client.js	requestAI	Selects Firebase AI Logic or configured API transport and validates output.
course-learning-engine.js	courseEligibility	Counts completed lessons, practice and passed tests; unlocks final and certificate.
assessment-engine.js	mountAssessmentCenter	Runs topic/final exams, times answers and records results.
aptitude-engine.js	recordAptitudeAttempt	Stores a bounded attempt summary plus per-question history and mistakes.
resume-engine.js	analyzeResume	Calculates evidence-based readiness and ATS metrics without inventing facts.
resume-builder.js	mountResumeBuilder	Builds editor, live preview, versions, export and Resume AI flow.
interview-engine.js	acceptTurn	Validates and appends one answer plus AI feedback/question.
interview-studio.js	mountInterviewStudio	Coordinates session, recognition, auto-submit, TTS, retry, pause and report.
speech-normalizer.js	assessSpeechReliability	Rejects low confidence, repeated, garbled or context-corrupt transcripts.
portfolio-engine.js	buildPortfolioEvidence	Combines course, exam, aptitude, resume, interview and project evidence.
career-roadmap-engine.js	careerEvidence	Maps the selected career role to verified core-skill and milestone evidence.
job-engine.js	calculateJobMatch	Scores supplied jobs using verified skill evidence and resume alignment.
platform-engine.js	buildSearchIndex	Creates one searchable index for courses, resources and saved content.
learning-store.js	saveLearningItem	Writes bounded teacher chats/notes to the signed-in user's subcollection.

Function contracts: input, output and dependencies

Function	Input	Output	Called from	Calls	Firebase / AI	Failure
requestMAGIVOXAI	operation, payload, user, signal	Validated AI result or error	Feature UI	requestAI	No direct Firestore; Gemini yes	Duplicate/quota/security/timeout errors are normalized
conversationHistory	messages and size limits	Bounded user/assistant messages	Teacher, Help, Resume	No external calls	No / No	Drops invalid roles, empty and excess text
courseEligibility	course, saved record, policy	Counts, percent, unlock/eligibility flags	Workspace, portfolio	Pure helpers	No / No	Missing data becomes zero evidence
recordAssessment	course record, result, policy	Normalized recorded attempt	Assessment callback	Mutates bounded record	Saved later to Firestore / No	Invalid or failed scores do not pass
analyzeResume	resume input, optional JD	Readiness, ATS, metrics, reasons, match	Resume builder, portfolio	normalizeResumeData, analyzeJobMatch	No / No	Incomplete data keeps score locked
mountResumeBuilder	workspace context	Mounted interactive UI	workspace.js	resume engine/store, AI Core	Firestore and Gemini as selected	Status retains form after errors
normalizeSpeechTranscript	raw transcript, profile name	raw/corrected/changes	Interview recording flow	guarded similarity and term rules	No / No	Uncertain text is not aggressively rewritten
assessSpeechReliability	raw, corrected, confidence, profile	reliable flag and reason	Interview studio	pattern checks	No / No	Unreliable answer is blocked before AI
acceptTurn	session, answer, AI result, source metadata	Updated session turn	Interview studio	normalizeTurn	Saved later; Gemini result already supplied	Invalid result is rejected
buildPortfolioEvidence	profile and appState	Readiness components, skills, next step	Progress, roadmap, jobs	course, resume, aptitude engines	No / No	Absent evidence scores zero
careerEvidence	profile, appState, career state	Role gaps, readiness, milestones	Career roadmap UI	portfolio engine, role map	No / No	Unknown role safely maps to a supported role
calculateJobMatch	job, profile, appState	Score or limited result plus gaps	Jobs UI	job analysis, portfolio evidence	No / No	Short JD returns limited/no fake score
saveLearningItem	user, kind, item, optional ID	Saved document ID	Teacher UI	account/shelf path helpers	Firestore yes / AI no	Offline or auth mismatch rejects write

Function	Input	Output	Called from	Calls	Firebase / AI	Failure
submitSupport	user, validated request, request ID	Reference ID	Help/Support UI	account and target selection	Firestore yes / AI no	Form stays visible and request ID is reusable safely

Part 7: Event Flow: From Button to Visible Result

General event sequence

User clicks -> Handler validates -> Engine transforms -> Store or AI runs -> UI redraws

A button is only the starting point. The event handler controls validation and busy state. A pure engine calculates or normalizes data. A store or AI service performs I/O. Finally, the feature redraws HTML or updates a status element.

Worked example: Save resume version

Verified source: **resume-builder.js**

```
root.querySelector('#saveResumeDraft').onclick = async () => {
  const saved = await saveResumeVersion(context.user, data());
  await context.saveResumeDraft?.(saved);
  await refreshVersions();
};
```

Stage	What happens
HTML	`#saveResumeDraft` is created in the resume builder template.
Event	The `.onclick` async handler runs once per click.
Data	`data()` normalizes all current resume form fields.
Firestore	`resume-store.js` writes `users/{uid}/resumes/{id}`.
Workspace state	The current draft is also saved through the workspace context.
Visible result	Status says the version was saved and the version list refreshes.

Major event flows

User action	Verified code flow
Signup	Submit form -> validate name/password -> create Firebase user -> set display name -> profile setup
Login	Submit -> choose persistence -> Firebase sign-in -> read user profile -> dashboard or profile setup
Logout	Click -> Firebase signOut -> remove MAGIVOX local keys -> index page
Change password	Submit -> validate strict policy -> reauthenticate current user -> updatePassword -> success status
Start course	Click course/topic -> courseRecord -> recordLessonOpen -> persist appState -> render lesson
Submit practice	Click -> read difficulty/answer -> recordPractice validates 8+ chars -> persist -> redraw
Complete lesson	Click -> canCompleteLesson -> completeLesson -> persist -> topic test becomes ready
Topic test	Open -> select 5 topic questions -> answer/timer -> score -> recordAssessment -> unlock on 70%
Final exam	Eligibility check -> 50 questions -> score -> record final -> certificate eligibility on 70%
Certificate	Eligible click -> create fact-bound ID -> publish exact public record -> preview/download
AI Teacher	Submit/quick action -> collect course/topic/recent history -> shared AI Core -> Gemini -> render/speak
Resume upload	Choose file -> validate size/type -> browser extractor -> editable text -> local analysis/optional AI
Interview	Start -> local opening question -> TTS -> mic -> stable final -> reliability -> AI turn -> TTS -> mic
Aptitude answer	Select answer -> deterministic validation -> feedback -> finish -> bounded attempt storage
Save project	Submit project facts -> normalize safe links -> save appState -> recalculate skill/readiness evidence
Career roadmap	Choose role -> map core courses -> build evidence -> milestones/plans -> save role and task state
Save job	Normalize supplied/sample job -> prevent duplicate -> save application record -> update overview

User action	Verified code flow
Application tracking	Change status/date/checklist -> updateApplication -> save appState -> redraw pipeline
Global search	Submit query/type -> build local index -> search/rank -> render open/save/note actions
Notifications	Open view -> derive from evidence/dates -> apply preferences/dismissals -> render and update badge

Part 8: Firebase Authentication

`firebase-auth.js` initializes one Firebase app and exports `app` and `auth`. Login, signup, logout, reset and route guards reuse this instance.

Email and Google login

Verified source: **firebase-auth.js**

```
const app = initializeApp(firebaseConfig);
const auth = getAuth(app);
export { app, auth };
```

Verified source: **firebase-auth.js**

```
await setPersistence(auth, browserLocalPersistence);
const result = await signInWithPopup(auth, googleProvider);
await redirectAfterLogin(result.user);
```

Profile-aware redirect

After authentication, `redirectAfterLogin` reads `users/{uid}`. A missing or incomplete profile goes to `profile-setup.html`; a completed profile goes to `dashboard.html`. A profile read failure does not incorrectly report bad credentials.

Password security

`password-policy.js` enforces at least eight characters, uppercase, lowercase, number and mandatory `@`. `account-security.js` reauthenticates the user before `updatePassword`, while reset email uses Firebase's reset workflow.

Action	Firebase API	Result
Google login	signInWithPopup	Authenticated user then profile redirect
Email signup	createUserWithEmailAndPassword + updateProfile	Account and display name created
Email login	signInWithEmailAndPassword	Session persistence then redirect
Forgot password	sendPasswordResetEmail	Firebase reset link sent
Logout	signOut	MAGIVOX local keys cleared and user returns to index
Change password	reauthenticateWithCredential + updatePassword	Current session proves identity before update

Part 9: Firestore Data Model and Persistence

Primary document model

Verified source: **workspace.js, learning-store.js, resume-store.js**

```
users/{uid}
  profile fields
  appState
    courseLearning, assessmentHistory, aptitudePractice
    resumeDraft, interviewSessions, projects, portfolio
    jobRecords, platformState, activityDates
  teacherChats/{id}
  teacherNotes/{id}
  resumes/{id}
```

Public and operational collections

Path	Who writes	Who reads	Purpose
users/{uid}	Owner	Owner or admin	Profile and consolidated app state
users/{uid}/teacherChats/{id}	Owner	Owner	Recent AI Teacher conversations
users/{uid}/teacherNotes/{id}	Owner	Owner	Saved teacher replies
users/{uid}/resumes/{id}	Owner	Owner	Up to 20 resume versions loaded
supportTickets/{id}	Signed-in submitting owner	Owner or admin	Problem/contact requests
featureRequests/{id}	Signed-in submitting owner	Owner or admin	Feature requests
certificates/{id}	Eligible learner	Public exact get; admin list	Certificate verification
publicPortfolios/{slug}	Portfolio owner	Public exact get; list denied	Recruiter portfolio page
adminContent/{id}	Admin	Admin	Draft/published content registry

State writes use `serverTimestamp()` for server-confirmed update time. Store functions verify that `auth.currentUser.uid` matches the requested user before building paths.

Firestore verbs in plain English

API	Meaning	Actual MAGIVOX use
collection	Points to a group of documents	Teacher chat/note shelves, admin queries
doc	Points to one exact document	User, certificate, support request, resume version
setDoc	Creates or replaces the supplied fields	App state, resume version, note, certificate, public portfolio
getDoc	Reads one exact document	Profile lookup and certificate verification
getDocs	Runs a bounded collection query	Resume versions, teacher library and admin lists
updateDoc	Changes selected fields	Admin resolves/reopens support status
deleteDoc	Deletes one permitted document	Resume versions, notes, admin content, public portfolio
serverTimestamp	Uses server-confirmed time	UpdatedAt/createdAt/publishedAt fields
onSnapshot	Live listener API	Not used in the inspected current data stores; screens explicitly reload/save
onAuthStateChanged	Authentication listener, not Firestore	Guards login, workspace and admin session state

Part 10: Firestore Security Rules

Owner and admin checks

Verified source: **firestore.rules**

```
function owner(userId) {
  return request.auth != null && request.auth.uid == userId;
}
function admin() {
  return request.auth != null && request.auth.token.admin == true;
}
```

Security decisions

- User profile and app state: owner create/update; owner or admin read.
- Teacher chats/notes and resume versions: strict allowed keys, length limits and server timestamps.
- Support submissions: owner UID must match authentication; initial status must be open.
- Certificate creation: ID format, score 70+, all five lessons/practices/topic tests and final evidence checked from the owner document.
- Certificates cannot be updated or deleted; exact public verification is allowed.
- Public portfolios deny list queries and validate public-safe fields.
- Admin content requires the Firebase custom claim `admin == true`.

The client interface is not the security boundary. Firestore rules remain the final authority even if somebody changes browser JavaScript.

Part 11: Shared MAGIVOX AI Core and Gemini

One operation registry

Verified source: **ai-core.js**

```
const OPERATIONS = Object.freeze({
  teacher: { path:'/chat', fields:['message','history','language', ...] },
  interviewTurn: { path:'/interview-turn', fields:['settings','turns', ...] },
  resumeAssist: { path:'/resume-assist', fields:['action','question','resume', ...] },
  careerJob: { path:'/career-job', fields:['action','question','job','evidence', ...] }
});
```

`scopedPayload` copies only fields approved for the active operation. `conversationHistory` keeps a bounded recent window. `requestKey` identifies duplicate in-flight requests. This gives all features the same concurrency and error behavior.

Firebase AI Logic client

Verified source: **ai-client.js**

```
firebaseContextPromise = Promise.all([
  import(FIREBASE_AI_URL), import('./firebase-auth.js')
]).then(([sdk, firebase]) => {
  const ai = sdk.getAI(firebase.app, { backend: new sdk.GoogleAIBackend() });
  return { sdk, ai };
});
```

The promise caches initialization. Each operation supplies a focused system instruction and, where needed, a response schema. AI inputs require a signed-in user and explicit consent.

Quota and failure handling

Verified source: **ai-client.js**

```
if (raw.includes('429') || raw.includes('quota')) {
  return classifiedAIError(
    'MAGIVOX AI is temporarily unavailable. Your work has been saved. Please retry later.',
    'MAGIVOX_AI_QUOTA'
  );
}
```

Feature	Operation	Relevant context only
Ask MAGIVOX AI / Teacher	teacher	Course, topic, level, language, mode, performance, recent conversation
Help Assistant	help	Question, language and recent help conversation
Resume AI	resumeAssist / resumeReview	Current resume, job description, action, question and recent resume conversation
Interview	interviewTurn / interviewReport	Settings, last turns, current answer and available audio assessment
Career/job AI	careerJob	One job, verified evidence summary, action and recent job conversation
Written exam review	examReview	Course, exam label and supplied written answers
Audio transcription	transcription	Recording, language, browser transcript and safe context

Part 12: Course and Lesson Flow

`course-catalog.js` defines the courses and five summary topics per course. `lesson-curriculum.js` expands each topic into beginner-to-advanced theory, examples, procedure, practice and checks. `workspace.js` renders the selected course and topic.

Required sequence

Open lesson -> Submit practice -> Mark complete -> Pass topic test 70% -> Unlock next topic

After all five topics pass, the 50-question final exam unlocks. A final score of 70% or more makes the learner certificate-eligible.

Verified source: **course-learning-engine.js**

```
export const COURSE_POLICY = Object.freeze({
  topicPass: 70, finalPass: 70,
  practiceDifficulties: ['Easy','Medium','Hard'],
  requiredPracticeAttempts: 1, durationHoursPerTopic: 4
});
```

What one lesson contains

Block	Purpose
Meaning/theory	Defines the concept in beginner language
Why it matters	Connects the topic to practical work
Workflow	Gives an ordered process
Worked example	Shows the concept in use
Common mistake	Warns about a realistic error
Verification	Explains how to check the result
Practice	Requires the learner to respond before completion
Topic test	Measures the topic and gates the next one

Part 13: Exams, Aptitude and Reasoning

Course assessments

`assessment-engine.js` selects topic-test or final-exam questions from the current course. Objective answers are scored locally. Written answers may use the `examReview` AI operation when consent and AI are available. Results are passed back to `recordAssessment` and stored in app state.

Aptitude engine

`aptitude-engine.js` generates fresh, seeded questions for aptitude and reasoning topics. It supports Topic Practice, Aptitude Test, Reasoning Test, Daily Challenge, Mock Exam and Practice My Mistakes. Hints, formula, shortcut and explanation remain local.

Verified source: **aptitude-engine.js**

```
return {
  attempts: state.attempts.length,
  accuracy: total ? Math.round(correct / total * 100) : 0,
  averageSeconds: total ? Number((seconds / total).toFixed(1)) : 0,
  strongTopics: ranked.filter(x => x.accuracy >= 70).slice(0,3),
  weakTopics: weak.filter(x => x.accuracy < 70).slice(0,3)
};
```

Integrity of scores

- Question IDs and answer results are recorded, not inferred from page clicks.
- Attempt histories are bounded to protect document size.
- Adaptive difficulty changes only after consistent evidence.
- AI does not replace deterministic objective marking.

Part 14: ATS Resume Studio

Two workflows

- Create My Resume: structured editor, live readiness checklist, templates, versions and exports.
- Review My Existing Resume: PDF/DOCX/TXT text extraction in the browser, editable extracted text and optional AI review.

Readiness gate

An ATS score is hidden until all six requirements are complete: contact details, target role, introduction, education, at least three relevant skills, and project/experience evidence.

Verified source: **resume-engine.js**

```
const atsScore = Math.round(
  formatting*.20 + grammar*.15 + skills*.20 + projects*.15 +
  contentQuality*.20 + keywordStrength*.10
);
```

Metric	How calculated locally
Formatting	Contact completeness, enough evidence bullets and reasonable extracted-text length
Grammar	Conservative text signals: lowercase starts, lowercase I, repeated spaces, long lines
Skills	Unique skill terms; no claim that a keyword proves competence
Projects	Presence, line count and truthful action-led evidence
Content quality	Required sections, evidence and avoidance of vague claims
Keyword strength	Overlap with the supplied job description or evidence terms when no JD exists

AI safety

Resume AI receives only the current normalized resume, optional JD, requested action/question and recent resume conversation. Its system instruction forbids invented qualifications, dates, metrics and achievements. Suggestions require the user to apply and verify them.

When Gemini is unavailable

The uploaded/extracted text and all structured form fields remain visible. `analyzeResume` and `analyzeResumeDocument` can still show evidence-based local readiness, missing sections, detected skills and JD overlap. The interface labels this as a local check and does not imitate an AI-written review.

Part 15: Voice-First Interview and English Coach

Modes and session model

`interview-engine.js` supports self-introduction, mock interview and English coaching. Modes include HR, Data Analyst, SQL, Power BI, Tableau, Java, Python, GIS and Web Development when mapped course data exists. Coaching supports Beginner, Intermediate and Advanced situations.

Natural voice cycle

Mic listens -> Final transcript -> Validate/normalize -> AI turn -> TTS speaks -> Listening resumes

Interim recognition text can be displayed for confidence, but it is never submitted. Auto-submit starts only after a stable final result and a short safe completion boundary. Busy, paused, muted, speaking and microphone states prevent overlaps and duplicate turns.

Speech accuracy pipeline

Verified source: **speech-normalizer.js**

```
export const SPEECH_LANGUAGE = 'en-IN';
export const LOW_RECOGNITION_CONFIDENCE = 0.6;
export const UNCERTAIN_RECOGNITION_CONFIDENCE = 0.75;
```

- Raw transcript is preserved exactly as returned by recognition.
- Final transcript contains stable recognition results only.
- Normalized transcript applies a saved first name only after `my name is` and only for a close match.
- Technical terms normalize only when skill/technology context exists.
- Low confidence, repeated words, profile-name mismatch, technical hallucination or garbled text stops automatic AI submission.
- The learner sees: `I may not have heard that correctly. Please say that again.` and can retry recording.

Verified source: **speech-normalizer.js**

```
logSpeechRecognition({ raw, final, corrected, language, confidence });
// Console labels: Raw transcript, Final transcript,
// Corrected transcript, Recognition language, Confidence
```

Multi-turn report

At least two answered questions are required. The final report averages assessed metrics, keeps mistakes and corrected sentences anchored to submitted text, adds vocabulary and filler feedback, and compares compatible prior attempts.

Metric	Evidence rule
Grammar, clarity, vocabulary, content, readiness	AI report validated against transcript turns
Fluency, confidence, pronunciation	Used only when a real audio assessment is available
Filler feedback	Counted deterministically from submitted answers
Overall score	Average of metrics that were actually assessed
History comparison	Same practice kind, mode, role, level and scoring version

Part 16: Progress, Portfolio and Readiness

`buildPortfolioEvidence` assembles evidence from all completed work. It does not claim that a resume keyword or course click proves skill. Course skill evidence combines lessons, practice, topic tests, final score and a small project-evidence bonus.

Overall readiness weights

Component	Weight	Source
Course progress	20%	Role-relevant course eligibility
Topic/final exams	10%	Saved assessment history
Aptitude and reasoning	10%	Per-question accuracy
Resume readiness	20%	ATS score only after all resume requirements
Interview practice	15%	Latest scored interview
Completed projects	15%	Projects with title, 30+ character description, tools and date
Role skill evidence	10%	Role-relevant skill evidence score

Verified source: **portfolio-engine.js**

```
const readiness = clamp(Object.values(components)
  .reduce((sum, x) => sum + x.score * x.weight / 100, 0));
```

Public portfolio

Private is the default. Publishing requires a valid unique-style slug and writes a sanitized public payload to `publicPortfolios/{slug}`. Firestore allows exact public reads and denies list access. Removing or changing a public slug unpublishes the old record.

Part 17: Career Roadmaps and Mentor Plans

Career roles map to explicit core courses, helpful courses, portfolio project ideas and interview topics. Current roles include Data Analyst, Business Analyst, GIS Analyst, Java Developer, Python Developer, Frontend Developer, Full Stack Developer, QA/Testing, Technical Support and Data Annotation.

Roadmap evidence

Verified source: **career-roadmap-engine.js**

```
roadmapReadiness = clamp(
  skillsScore*.45 + projectScore*.15 + resumeScore*.15 +
  interviewScore*.15 + aptitudeReasoning*.10
);
```

Milestones

Foundation	->	**Core Skills**	->	**Practice**	->	**Projects**	->	**Resume**	->	**Interview**	->	**Job Preparation**

Daily plans contain two to four tasks depending on available study minutes. Weekly plans assign a theme for each day. Plans are generated from missing evidence, then users mark tasks done, skipped or rescheduled.

Part 18: Jobs and Application Tracking

The Jobs workspace separates sample workflows from real listings. If `jobsBaseUrl` is empty, only clearly labelled MAGIVOX sample jobs appear. Users can paste a real job and official HTTPS link. When configured, the backend provider is Adzuna.

Evidence match formula

Component	Weight
Verified skills	55%
Target role	15%
Relevant projects	10%
Certificates	10%
Resume alignment	10%

A detailed match is returned only when the supplied description contains at least 25 words. Otherwise the result says it is limited and does not pretend to know requirements.

Safety and tracking

- `jobSafety` flags payment requests, guarantees, unofficial-only contact, unclear employers and shortened/message links.
- Statuses are Saved, Applied, Assessment, Interview, Offer, Selected and Rejected.
- The tracker stores notes, dates and a preparation checklist. MAGIVOX does not submit applications.
- Career AI receives one job and a compact verified evidence summary; it cannot search live jobs itself.

Part 19: Search, Saved Items, Notes, Flashcards and Notifications

Unified local search

`buildSearchIndex` creates a search index for courses, lessons, PDFs, practice, questions, jobs, projects, notes and help. `searchPlatform` filters and ranks it locally, so search remains useful when Gemini is unavailable.

Notes and AI

Manual notes save immediately through workspace state. `Create notes with AI`, `AI flashcards` and `AI quiz` call the shared Teacher operation with the note title and supplied note content. The generated response is shown as a draft. Flashcards are saved only when Question/Answer pairs can be parsed.

Notifications

Notifications are derived from real dates and progress: unfinished learning, interviews, follow-ups and other evidence. Users can mark read/unread, dismiss items and change preferences. These reminders are local application logic, not push notifications.

Current evidence	->	deriveNotifications	->	Preference filter	->	Badge/list	->	User action

Part 20: Admin System: Current Capability and Limits

`admin.js` checks a freshly refreshed Firebase ID token and requires `token.claims.admin === true`. It then loads bounded user, certificate, support, feature-request and admin-content queries.

Implemented

- Platform metrics: total users, 30-day active learners, course completions, average exam, certificates, support, resume and interview usage.
- Content records: Course, Module, Topic, PDF/resource, questions, projects, jobs and Help/FAQ metadata.
- Create, edit and delete `adminContent` records.
- Resolve or reopen support and feature requests.

Honest limitation

The deployed course catalog, lessons, PDFs and question banks still come from static source files. `adminContent` is an authenticated content registry, but the learner course renderer does not currently replace those static files with published admin records. An admin edit therefore does not automatically change live course content.

Part 21: Error Handling and Recovery

Failure	Handling	User work
Gemini quota/429	One consistent unavailable message; no automatic retry loop	Input, current form and conversation stay in place
Duplicate AI request	`activeRequests` rejects identical in-flight request	Original request remains active
AI timeout	85-second timeout and friendly message	User can retry
Offline Firestore save	Store rejects before write	Editor content remains visible
Auth profile read failure	Login still succeeds; destination fallback used	Session is not falsely rejected
Speech low confidence	No AI submission; retry prompt	Raw/final/normalized transcript retained
Invalid job URL	Only clean HTTPS URL accepted	Form remains available
PDF/DOCX extraction issue	Clear extracted-text error and editable fallback	Original local file is not uploaded by extraction alone
Permission denied	Generic safe message; Firestore rules deny operation	No unauthorized data is returned

Retry principle

Retry is user-controlled. The system does not repeatedly spend quota or create duplicate writes. Busy flags, request IDs and in-flight keys make each action explicit.

Part 22: Dependency and Data-Flow Diagrams

Authenticated workspace

onAuthStateChanged -> Load users/{uid} -> Normalize appState -> Mount active tool -> Save patch

AI request

Feature context -> requestMAGIVOXAI -> requestAI -> Firebase AI Logic -> Gemini response

Course certificate

5 lessons -> 5 practices -> 5 topic tests 70% -> Final 70% -> Public certificate

Interview answer

en-IN recognition -> Final-only transcript -> Reliability check -> AI follow-up -> TTS then mic

Job preparation

Supplied job/JD -> Deterministic match -> Evidence gaps -> Preparation pack -> Optional Career AI

Module dependency table

Feature UI	Imports engines/services	Writes/reads
teacher.js	ai-core, course-catalog, teacher-conversation, voice-recorder, learning-store	Teacher chat/note subcollections and hosted AI
interview-studio.js	interview-engine, speech-normalizer, voice-recorder, ai-core	Workspace interviewSessions and hosted AI
resume-builder.js	resume-engine, resume-store, ai-core, docx-worker	Resume subcollection, draft app state and AI
jobs.js	job-engine, ai-core, ai-client	Workspace jobRecords; optional live provider and AI
progress-portfolio.js	portfolio-engine, portfolio-store, ai-core	Workspace projects/portfolio and public portfolio
platform-hub.js	platform-engine, ai-core	Workspace platformState and optional AI

Part 23: Beginner Reading and Learning Plan

You do not need to memorize every line. Learn one vertical flow at a time: HTML control, event handler, engine function, saved data and visible result.

Week	Focus	Practical goal
1	HTML pages	Open index/login/workspace HTML. Identify forms, IDs, links and the workspace container.
2	CSS basics	Learn box model, grid, flex, media queries. Trace one card from HTML to its selector.
3	JavaScript DOM	Practice querySelector, events, preventDefault, template strings and class toggles.
4	Modules and state	Trace imports from workspace.js to one feature mount and engine.
5	Firebase Auth	Explain initializeApp, getAuth, auth-state guard, login, logout and profile redirect.
6	Firestore	Trace one appState save, one resume version save and one security rule.
7	Course system	Explain recordLessonOpen through certificate eligibility.
8	AI system	Explain operation registry, scoped payload, Gemini request, consent and quota failure.
9	Interview voice	Trace recognition final result, normalization, reliability, AI, TTS and resume listening.
10	Career features	Study ATS, portfolio readiness, roadmap and job formulas.
11	Testing	Read tests beside each engine and run npm test/check/build.
12	Interview practice	Answer Part 24 aloud and demonstrate one feature end-to-end.

How to explain any feature

- Start with the user problem.
- Name the page and mount function.
- Name the event handler and data collected.
- Explain deterministic rules before remote services.
- Name the Firestore path or AI operation.
- Finish with visible success and failure behavior.

Part 24: Project-Specific Interview Questions and Answers

This section contains 60 project-specific questions. Practise speaking each answer in your own words, then demonstrate the related source file.

Q1. What is MAGIVOX AI?

Answer: A browser-based learning and career platform that combines structured courses, assessments, voice interview practice, resume tools, job preparation and evidence-based progress. Firebase provides identity and persistence; Gemini provides genuine generative coaching.

Q2. Why is the app modular?

Answer: UI modules mount each feature, pure engines hold testable rules, and store/service modules handle external I/O. This reduces duplication and makes behavior easier to verify.

Q3. How does workspace routing work?

Answer: The `tool` query parameter selects the active feature. `workspace.js` validates the value, highlights the sidebar link and calls the corresponding mount function.

Q4. Why use dynamic HTML inside mount functions?

Answer: The shell stays stable while each tool owns its interface and listeners. It also avoids loading every feature's visible DOM at once.

Q5. What is the shared AI Core?

Answer: `ai-core.js` maps named operations to allowed fields and paths, trims context, prevents duplicate requests, and standardizes errors before calling `ai-client.js`.

Q6. How is duplicate AI work prevented?

Answer: A request key combines user, operation and compact payload. Identical in-flight work is rejected until the original promise finishes.

Q7. How is AI initialization reused?

Answer: `ai-client.js` caches `firebaseContextPromise`, so Firebase AI Logic and the shared Firebase app are initialized once and reused.

Q8. Why is AI context scoped?

Answer: Each tool sends only relevant data. This reduces privacy exposure, tokens, latency and prompt confusion.

Q9. What happens when Gemini quota is exhausted?

Answer: The error is classified as `MAGIVOX_AI_QUOTA`; the consistent message says MAGIVOX AI is temporarily unavailable and work is saved. No fake generated answer or retry loop is used.

Q10. Does MAGIVOX use local fake AI?

Answer: No for features described as generated AI. Local deterministic logic is used for scoring, eligibility, search, navigation, stored quizzes and safety checks.

Q11. How are conversations maintained?

Answer: Recent user/assistant messages are validated and trimmed by `conversationHistory`, then supplied with the active course, resume, interview or job context.

Q12. How does Telugu work?

Answer: The teacher operation accepts English, Telugu, Telugu in English letters and Hindi. The selected language becomes part of the system instruction; technical terms remain accurate.

Q13. How does login work?

Answer: `firebase-auth.js` supports Google popup and email/password. Persistence is set, then the user document decides whether profile setup or the dashboard opens.

Q14. How is password change secured?

Answer: The user reauthenticates with the current password before Firebase `updatePassword` runs, and the same strict password policy validates the new value.

Q15. What is stored in users/{uid}?

Answer: Profile fields and consolidated appState: course progress, assessments, aptitude, current resume, interviews, projects, portfolio, jobs, platform state and activity dates.

Q16. Why are resumes also a subcollection?

Answer: The appState holds the active draft, while `users/{uid}/resumes/{id}` supports named versions, sorting and deletion without expanding the main document unnecessarily.

Q17. How do Firestore rules enforce ownership?

Answer: The `owner(userId)` helper requires an authenticated UID equal to the path UID. Every private user path uses it.

Q18. How are public certificates safe?

Answer: Creation requires a strict ID and fields, score 70+, and server-checked evidence that all five lessons, practices, topic tests and final exam were completed. Updates and deletes are denied.

Q19. Why deny listing public portfolios?

Answer: Recruiters can open a known slug, but cannot enumerate every learner portfolio through Firestore.

Q20. What is the course unlock sequence?

Answer: Open lesson, submit one practice answer, mark the lesson complete, score at least 70% in the topic test, then the next topic unlocks.

Q21. How is final exam access controlled?

Answer: All topics require completed lessons, practice and passed topic tests. The final exam then unlocks; 70% passes.

Q22. How is course progress calculated?

Answer: Earned units are completed lessons plus practices plus topic tests plus passed final, divided by three units per topic plus one final unit.

Q23. What does a complete lesson teach?

Answer: Definition, why it matters, workflow, example, mistake, verification and practice, followed by an assessment.

Q24. How does aptitude practice work?

Answer: The engine generates validated local questions by category, topic and difficulty. Attempts include accuracy, time and per-question results.

Q25. How are weak topics found?

Answer: Performance analytics groups question history by topic and marks accuracy below 70% as weak evidence.

Q26. Why is the ATS score hidden initially?

Answer: A numeric score without enough data is misleading. Six essential resume requirements must be complete first.

Q27. What is the ATS score formula?

Answer: 20% formatting, 15% grammar, 20% skills, 15% projects, 20% content quality and 10% keyword strength.

Q28. Can Resume AI invent achievements?

Answer: No. The prompt forbids invented qualifications, grades, dates, jobs, skills and metrics, and suggestions require user verification before applying.

Q29. How are uploaded resumes handled?

Answer: PDF, DOCX or TXT text is extracted in the browser. The user reviews editable text before requesting AI. Visual layout cannot be reliably evaluated from extracted text.

Q30. How does voice-first interview work?

Answer: Recognition listens in en-IN, collects final results, validates and safely normalizes them, auto-submits the answer, waits for the AI response, speaks it with TTS, then resumes listening.

Q31. Why never submit interim speech?

Answer: Interim hypotheses change while the user speaks. Submitting them early corrupts context and can create duplicate turns.

Q32. How are names corrected safely?

Answer: Only after a clear `my name is` pattern, and only when the heard token is a close edit-distance match to the saved first name.

Q33. Why not normalize every technical word?

Answer: Aggressive replacement can turn ordinary speech into Java/CSS words. Term normalization runs only when the sentence has skill or technology context.

Q34. When is a transcript rejected?

Answer: Below 0.6 confidence, too little speech, strong repetition, profile-name mismatch, technical hallucination or garbled character ratio.

Q35. How does the app avoid listening to its own voice?

Answer: Recognition is stopped before TTS. The state shows AI Speaking, and listening restarts only after speech completion when not paused or mic-disabled.

Q36. What if TTS fails?

Answer: The response remains visible as text and the conversation can continue safely.

Q37. How is interview scoring fair?

Answer: Text metrics come from validated transcript content. Audio metrics are included only when real audio assessment exists. Obvious recognition errors and regional accents are not penalized.

Q38. Why require two answers for a report?

Answer: A single answer is too little evidence for a multi-metric interview estimate. The engine throws until at least two completed answers exist.

Q39. How is improvement compared?

Answer: Only attempts with the same kind, mode, role, level and rubric version are compared.

Q40. What is the portfolio readiness formula?

Answer: Courses 20%, exams 10%, aptitude/reasoning 10%, resume 20%, interview 15%, completed projects 15%, role skills 10%.

Q41. How is skill evidence calculated?

Answer: Per course: lesson ratio times 30, test ratio times 40, final score times 0.2 and a 10-point relevant completed-project bonus, capped at 100.

Q42. What makes a project completed?

Answer: A title, description of at least 30 characters, tools and a completion date.

Q43. How are career roadmaps personalized?

Answer: The selected role maps to core skills. Existing course, project, resume, aptitude and interview evidence chooses the next missing milestone.

Q44. What is roadmap readiness?

Answer: 45% core skills, 15% projects, 15% resume, 15% interview and 10% aptitude/reasoning.

Q45. Are jobs live?

Answer: Only if `jobsBaseUrl` connects the provider. Otherwise clearly labelled sample jobs support workflow practice, and users may add real links manually.

Q46. How is a job match calculated?

Answer: For a full JD: verified skills 55%, target role 15%, projects 10%, certificates 10% and resume alignment 10%.

Q47. Why require a full job description?

Answer: Without enough supplied text, the app cannot honestly claim complete requirements or a detailed match.

Q48. What job-scam signals are checked?

Answer: Payment requests, unrealistic guarantees, unofficial-only contact, unclear company identity and shortened or messaging links.

Q49. Does MAGIVOX apply for jobs?

Answer: No. It supports discovery, evidence comparison, preparation and a private tracker; the learner controls external applications.

Q50. How does global search work without AI?

Answer: A local index combines course, resource, saved and help metadata, then filters and ranks by query and content type.

Q51. How do AI notes work?

Answer: The current note text and title are sent through the shared Teacher operation with a concise instruction. The result is a draft and does not overwrite the note automatically.

Q52. What can the admin panel change?

Answer: It manages admin content records and support status, and displays metrics. It does not yet replace static live course files.

Q53. How are cloud requests validated?

Answer: The optional Function handler validates origin, method, auth token, non-anonymous user, pilot access, JSON type, payload size, consent, route input and quota before the provider call.

Q54. How is sensitive logging avoided?

Answer: The handler logs only a generic error type, never raw questions, resumes, recordings, tokens, provider responses or keys.

Q55. How are limits applied in the optional backend?

Answer: Per-minute user and global units plus daily user and global units are checked before consuming request cost.

Q56. How is the project tested?

Answer: `npm test` runs the syntax/import check and Node tests for engines, client, learning security and Function routes. `npm run build` copies validated public files to dist-pages.

Q57. What would you improve next?

Answer: Connect published admin content to a versioned learner content loader, add a production job provider only after credentials and monitoring, and add browser automation for authenticated voice/device scenarios.

Q58. What was your role as project owner?

Answer: I defined the product goals, workflows, content needs and acceptance behavior, reviewed outputs and tested the product. Codex assisted with implementation; I should explain only code and design decisions I have personally studied and verified.

Q59. How would you debug a broken button?

Answer: Inspect its HTML ID/data attribute, find the bound listener, confirm validation and busy state, trace engine/service calls, inspect the status element, then check console, network and Firestore rules.

Q60. How would you explain the architecture in 30 seconds?

Answer: MAGIVOX is a modular browser app. Workspace routes mount feature UIs, pure engines calculate learning and career evidence, Firebase Auth and Firestore secure user data, and one shared AI Core sends focused context to Gemini for conversational features.

Part 25: How to Describe Your Role Honestly

A strong interview explanation separates product ownership from implementation assistance. Do not claim that you personally typed or designed code you have not studied.

Recommended answer

I am the project owner of MAGIVOX AI. I defined the learner problems, prioritized the features, described the required behavior, supplied content and visual references, reviewed each implementation and tested the user flows. I used Codex as an AI coding assistant to help implement and refine the code. I am now studying the HTML, CSS, JavaScript, Firebase and AI architecture so I can explain how each feature works and continue maintaining it responsibly.

If asked what you coded

Answer specifically. For example: `I understand and can explain the course progression engine and Firestore rules. Codex assisted with the original implementation, and I verified the behavior with tests.` If you later modify a feature yourself, explain that exact contribution.

Evidence you can show

- Your master requirements and acceptance cases.
- The running MAGIVOX workflows you reviewed.
- This architecture guide and the source files you studied.
- Test commands and results you can reproduce.
- A small change you can implement and explain independently.

Part 26: Complete Code Summary, Major Buttons and Top Files

Every major button: code-to-result map

Area	Button/action	HTML origin	Handler/function	Data	Remote call	Visible result
Landing	Continue as guest	index.html #guestButton	script.js click handler	Sets guest name/mode in localStorage	No	Dashboard opens in guest mode
Login	Continue with Google	login.html #googleButton	firebase-auth.js click	Google identity/session	Auth + profile read	Profile setup or dashboard
Login	Log in	login.html #emailLoginForm	firebase-auth.js submit	Email, password, persistence choice	Firebase Auth	Dashboard/profile setup or error
Login	Forgot password	login.html reset link/form	firebase-auth.js reset handler	Email address	Firebase Auth	Reset email status
Signup	Create account	signup.html #emailSignupForm	firebase-auth.js submit	Name, email, validated password	Firebase Auth	Profile setup
Global	Log out	workspace/dashboard #logoutButton	firebase-auth.js click	Current session/local MAGIVOX keys	Firebase Auth	Index page
Workspace	Sidebar tool links	workspace.html data-tool-link	normal navigation + workspace.js	tool query parameter	Auth state/profile load	Selected tool mounts
Courses	Open/Continue lesson	dynamic course card	workspace.js course click	Course ID, topic index	Firestore appState	Lesson and progress shown
Courses	Submit practice	dynamic practice form	workspace.js submit	Answer and difficulty	Firestore appState	Practice recorded
Courses	Mark lesson complete	dynamic button	workspace.js click -> completeLesson	Opened lesson + practice evidence	Firestore appState	Topic test unlocks
Courses	Start topic test	dynamic link/button	assessment-engine.js	Course and topic question set	No AI unless written review	Timed test opens
Courses	Submit exam	assessment form	assessment-engine.js submit	Answers, time, score	Firestore appState; optional AI written review	Score, breakdown, next action
Courses	Certificate	dynamic eligible action	workspace.js -> createCertificateRecord/publishCertificate	Verified course record	Firestore certificates	Certificate preview/download
Aptitude	Start practice	#startAptitude	aptitude-studio.js click	Mode, category, topic, difficulty, count	No	Question session begins
Aptitude	Hint	dynamic hint button	aptitude-studio.js click	Current question hint	No	Hint appears
Aptitude	Submit/Finish	dynamic form/button	aptitude-studio.js -> buildAttempt/recordAptitudeAttempt	Answers and time	Firestore appState	Score and analytics update
Teacher	Quick actions	teacher.js data-action	click -> requestSubmit	Current course/topic and direct action prompt	Gemini via shared AI Core	AI reply appears
Teacher	Send message	#teacherForm	teacher.js submit	Question, scoped learning context, recent history	Gemini via shared AI Core	Conversational answer
Teacher	Retry	#retryTeacher	teacher.js click	One saved failed request	Gemini once	Reply or consistent error
Teacher	Read aloud/Stop	#autoRead/#stopReading	teacher.js/studio-utils.js	Latest AI text	Browser TTS	Speech starts/stops
Teacher	Save chat	#saveChat	teacher.js -> saveLearningItem	Recent bounded messages/context	Firestore teacherChats	Saved-chat list refreshes
Teacher	Save note	#saveNote	teacher.js -> saveLearningItem	Latest AI reply	Firestore teacherNotes	Saved-note list refreshes
Help	Quick topic	data-help	help-support.js click -> requestSubmit	Built-in FAQ question	Optional shared AI	Help answer appears
Help	Get help	#helpForm	help-support.js submit	Question and optional recent history	Optional Gemini; local guide without AI	Answer/status shown
Support	Submit request	#supportForm	help-support.js -> submitSupport	Validated kind/category/contact/title/description	Firestore support/feature collection	Reference ID shown
Resume	New	#newResume	resume-builder.js click	Profile defaults	No	Fresh draft appears
Resume	Save version	#saveResumeDraft	resume-builder.js -> saveResumeVersion	Normalized form data	Firestore resume subcollection + appState	Version refresh/status
Resume	Duplicate/Rename/Delete	three builder buttons	resume-builder.js handlers	Current version metadata	Firestore for save/delete	Version changes

Area	Button/action	HTML origin	Handler/function	Data	Remote call	Visible result
Resume	Print/Save PDF	#downloadPDF	resume-builder.js -> printResume	Rendered normalized resume	No	Browser print dialog
Resume	Download DOCX	#downloadDOCX	resume-builder.js -> exportDocx	Normalized resume	No	DOCX download
Resume	Tailor resume	#tailorResume	resume-builder.js -> tailorResumeToJob	Resume skills + supplied JD	No	Existing relevant items reorder
Resume	Run Resume AI	#runResumeAI	resume-builder.js -> requestMAGIVOXAI	Action, resume, JD, question, history	Gemini	Suggestion requiring user apply
Interview	Start	#startInterview	interview-studio.js click -> createInterviewSession	Practice settings	No - openingQuestion is local	Question shown/spoken; later turns use Gemini
Interview	Mic on/off	recorder data-record / interview control	voice-recorder.js + interview studio	Media stream and recognition	Browser SpeechRecognition	Listening state/transcript
Interview	Finish answer	recorder data-stop	voice-recorder stopAndTranscribe	Recorded speech	Recognition/transcription path	Final validated answer auto-submits
Interview	Send typed answer	#interviewAnswerForm	interview-studio.js submit	Typed fallback	Gemini	Next response/question
Interview	Pause/Resume	#pauseInterview	interview-studio.js click	Session phase, mic and TTS	No	Conversation pauses/resumes
Interview	Retry Answer	#retryInterviewAnswer	interview-studio.js -> retryLastAnswer	Last turn removed/replaced	Gemini after new answer	Corrected transcript history
Interview	Skip question	#skipInterview	interview-studio.js -> skipTurn	Current question marked skipped	Gemini	One next question
Interview	Mute AI	voice toggle	interview-studio.js	TTS preference	Browser TTS	Voice silenced; text retained
Interview	End Interview	#endInterview	interview-studio.js -> report operation	All validated turns	Gemini report + Firestore appState	Full report/history
Roadmap	Save/update plan	career form/actions	career-roadmaps.js -> career engine	Role, profile, days, study minutes	Firestore appState	Milestones and plan redraw
Roadmap	Task done/skip/reschedule	dynamic mentor controls	career-roadmaps.js -> applyMentorTask	Task ID and status/date	Firestore appState	Daily plan updates
Jobs	Search/Clear	#jobsFilterForm/#clearJobFilters	jobs.js -> searchJobs	Local filters	No	Filtered sample/supplied jobs
Jobs	Search live provider	#searchLiveProvider	jobs.js -> requestAI('/jobs')	Query and up to three cities	Firebase Auth + configured provider	Adzuna-labelled results
Jobs	Save job	dynamic data-save-job	jobs.js -> context.saveJob	Normalized job	Firestore appState	Saved status
Jobs	Add job link	#manualJobForm	jobs.js submit	Real job, safe HTTPS URL and JD	Firestore appState	Private tracker item
Jobs	Prepare Me	dynamic detail	jobs.js -> preparationPack	Job and verified evidence	No	Skills, questions and 3/7-day plans
Jobs	Ask Career AI	#askJobAI	jobs.js -> requestMAGIVOXAI	One job + compact evidence	Gemini	Contextual job guidance
Library	Search	#globalSearchForm	platform-hub.js -> searchPlatform	Query/type and local index	No	Ranked content results
Library	Save/Unsave	data-save-result	platform-hub.js -> toggleSaved	Selected indexed item	Firestore appState	Saved library changes
Notes	Save note	#noteForm	platform-hub.js submit	Normalized title/topic/content	Firestore appState	Note card appears
Notes	Create notes with AI	#aiImproveNote	platform-hub.js -> noteAI	Current source note	Gemini Teacher operation	Draft answer shown
Notes	AI flashcards/quiz	data-note-cards/data-note-quiz	platform-hub.js -> noteAI	Saved note content	Gemini Teacher operation	Draft; parsed cards may save
Flashcards	Show/rate/delete	data card actions	platform-hub.js handlers	Card ID/status	Firestore appState	Card updates
Notifications	Mark all/read/delete	notification actions	platform-hub.js handlers	Derived notification state	Firestore appState	Badge/list updates
Portfolio	Save project	project form	progress-portfolio.js -> normalizeProject	Project facts and safe links	Firestore appState	Evidence/readiness recalculates
Portfolio	AI evidence help	#runPortfolioAI	progress-portfolio.js -> requestMAGIVOXAI	Saved evidence facts only	Gemini Teacher operation	Truthful draft response
Portfolio	Publish	#savePortfolioSettings	progress-portfolio.js -> publishPublicPortfolio	Sanitized public payload	Firestore publicPortfolios	Recruiter link or private status
Portfolio	Preview/Print	#previewRecruiter/#printPortfolio	progress-portfolio.js handlers	Current portfolio evidence	No	Preview or print dialog
Settings	Save profile	settings form	workspace.js save handler	Validated profile fields	Firestore users/{uid}	Identity/navigation refresh
Settings	Change password	security form	account-security.js -> changePassword	Current/new passwords	Firebase Auth	Success or reauth error
Admin	New/Save/Edit/Delete	admin controls	admin.js handlers	Validated content record	Firestore adminContent	Admin list reloads

Area	Button/action	HTML origin	Handler/function	Data	Remote call	Visible result
Admin	Resolve/Reopen	data-request-status	admin.js -> updateDoc	Only status field	Firestore support/feature collection	Request list reloads

Complete important-file summary

File	Purpose	Main functions	Used by	Firebase?	AI?	Difficulty
workspace.html	Authenticated shell	Static IDs/navigation	All workspace tools	Auth load	No	Beginner
workspace.js	Tool routing and shared state	renderActiveTool, showCourse, changeWorkspaceState	All workspace tools	Firestore	No direct	Advanced
workspace.css	Workspace presentation	Selectors/media queries	Workspace and dynamic UI	No	No	Intermediate
studio.css	AI/resume/interview presentation	Studio selectors	Teacher, resume, interview	No	No	Intermediate
firebase-auth.js	Identity/session	redirectAfterLogin and event handlers	Auth pages and guards	Auth + profile get	No	Intermediate
ai-core.js	Shared AI boundary	conversationHistory, requestMAGIVOXAI	All generative features	Auth user passed	Yes	Advanced
ai-client.js	Gemini prompts/transport	requestAI, friendlyAIError, modelFor	AI Core	Firebase AI Logic	Yes	Advanced
learning-store.js	Learning/support/certificate I/O	saveLearningItem, submitSupport, publishCertificate	Teacher, Help, Course	Firestore	No	Intermediate
course-catalog.js	Course/topic source	getCourse, courseSearchText	Course, Teacher, search, jobs	No	No	Beginner
lesson-curriculum.js	Detailed lesson matter	getTopicCurriculum	Course lesson UI	No	No	Intermediate
course-learning-engine.js	Progress state machine	recordPractice, completeLesson, courseEligibility	Course, portfolio	Saved through workspace	No	Advanced
assessment-engine.js	Exam runtime	mountAssessmentCenter	Exams	State save	Optional written marking	Advanced
aptitude-engine.js	Question/scoring engine	generateQuestion, recordAptitudeAttempt, performanceAnalytics	Aptitude UI, portfolio	State save later	No	Advanced
aptitude-studio.js	Aptitude interface	mountAptitudeStudio	Workspace	State callback	Teacher links only	Advanced
teacher.js	AI Teacher interface	mountTeacher	Ask MAGIVOX AI	Chats/notes	Yes	Advanced
teacher-conversation.js	Conversation rules/render helpers	teacherHistory, answerHTML	Teacher/AI client	No	Prompt guidance	Intermediate
voice-recorder.js	Microphone/recognition component	mountRecorder	Teacher/interview	No	Transcription optional	Advanced
speech-normalizer.js	STT safety	normalizeSpeechTranscript, assessSpeechReliability	Interview	No	No	Advanced
interview-engine.js	Session/report model	createInterviewSession, acceptTurn, normalizeReport	Interview studio/AI client	Saved later	Schemas validate AI	Advanced
interview-studio.js	Voice-first coordinator	mountInterviewStudio, submit, speakThenListen	Interview tool	App state	Yes	Advanced
resume-engine.js	ATS and JD rules	analyzeResume, analyzeJobMatch	Resume, portfolio, jobs	No	No	Advanced
resume-builder.js	Resume creation UI	mountResumeBuilder, draw, printResume	Resume tool	Firestore versions	Yes	Advanced
resume-reader.js	Browser file extraction	PDF/DOCX/TXT readers	Existing-resume review	No	No	Advanced
resume-store.js	Resume version I/O	list/save/deleteResumeVersion	Resume builder	Firestore	No	Intermediate
portfolio-engine.js	Readiness/evidence	buildPortfolioEvidence, publicPortfolioPayload	Progress, roadmap, jobs	No direct	No	Advanced
progress-portfolio.js	Portfolio UI	mountProgress	Progress tool	App/public Firestore	Yes	Advanced
career-roadmap-engine.js	Role/gap plans	careerEvidence, smartRecommendation	Roadmap and jobs	No direct	No	Advanced
career-roadmaps.js	Roadmap UI	mountCareerRoadmaps	Roadmaps	App state	Yes	Advanced
job-engine.js	Job matching/tracking rules	calculateJobMatch, preparationPack, jobSafety	Jobs UI	No direct	No	Advanced
jobs.js	Jobs interface/provider calls	mountJobs, searchLive, runCareerAI	Jobs tool	App state/Auth	Yes	Advanced
platform-engine.js	Search/library rules	buildSearchIndex, searchPlatform, deriveNotifications	Platform hub	No direct	No	Intermediate

File	Purpose	Main functions	Used by	Firebase?	AI?	Difficulty
platform-hub.js	Library interface	mountPlatformHub, noteAI	Search & Saved	App state	Yes	Advanced
help-support.js	Help and support UI	mountSupport	Help tool	Support collections	Optional	Intermediate
portfolio-store.js	Public portfolio I/O	publish/unpublish/read helpers	Progress/public page	Firestore	No	Intermediate
admin.js	Admin dashboard	load, drawItems, edit	admin.html	Firestore + claims	No	Advanced
firestore.rules	Server authorization	owner/admin/validators	All Firestore operations	Security layer	No	Advanced
functions/handler.js	Optional secured API router	createHandler	Cloud Functions	Token verification/limits	Provider route	Advanced
scripts/check.mjs	Syntax/import checker	check traversal	npm test/check	No	No	Intermediate
scripts/build.mjs	Deployable copy builder	build traversal/copy	npm run build	No	No	Intermediate

Verified quality state while preparing this guide

The existing automated suite completed successfully: 103 tests passed, 0 failed. The project syntax checker validated 69 first-party JavaScript files. The documentation generator did not modify application source files.

Top 20 files to understand first

Order	File	Why it matters
1	workspace.js	Authenticated state, tool router and course UI
2	workspace.html	Stable shell and navigation
3	ai-core.js	Shared AI operation and request control
4	ai-client.js	Gemini/Firebase AI Logic prompts, schemas and transport
5	firebase-auth.js	Firebase app, login, signup, logout and reset
6	firestore.rules	Production authorization and data validation
7	course-catalog.js	Course/topic metadata and resources
8	lesson-curriculum.js	Step-by-step lesson matter
9	course-learning-engine.js	Progress gates and certificates
10	assessment-engine.js	Topic and final exam runtime
11	teacher.js	Conversational learning workspace
12	interview-studio.js	Voice-first UI state machine
13	interview-engine.js	Session/report validation
14	speech-normalizer.js	en-IN normalization and reliability
15	resume-builder.js	Resume editor, versions, export and AI
16	resume-engine.js	ATS/readiness rules
17	portfolio-engine.js	Evidence and readiness aggregation
18	career-roadmap-engine.js	Role milestones and plans
19	job-engine.js	Job normalization, matching, safety and preparation
20	platform-hub.js	Search, saved, notes, flashcards and notifications

Final architecture summary

Question	Answer
Where does UI start?	Static page plus a feature mount function.
Where are business rules?	Pure engine modules with exported functions and tests.
Where is private data?	Owner-scoped Firestore documents and subcollections.

Question	Answer
Where is generative AI?	One shared AI Core calling Gemini through Firebase AI Logic.
What continues without AI?	Courses, local exams, aptitude, ATS calculations, search, progress, job tracker and manual notes.
What remains external?	Gemini quota, Firebase configuration, App Check/authorized domains and optional live jobs provider.
Main current limitation?	Admin content records do not yet replace static learner content.
Best interview demo?	Login -> open SQL lesson -> complete practice/test -> ask a follow-up in Teacher -> show saved progress and evidence.